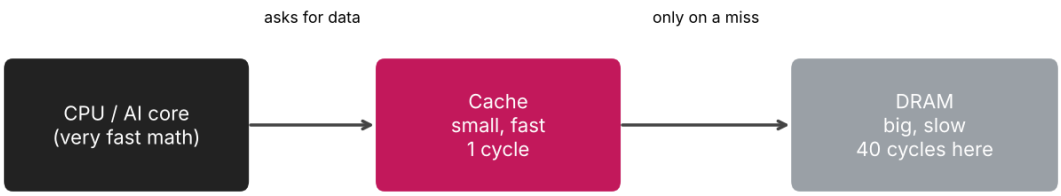
AI-Aware Cache & Prefetch Engine

Micron RTL Challenge - complete learning guide



Prefetch engine: brings data from DRAM into the cache BEFORE the CPU asks

Result: 2.26x faster on average (geometric mean, 9 workloads), 2.06x on the AI-inspired workloads, never slower than a normal cache.

Kotapati Kishore | 1 November 2026

Everything in this guide uses numbers measured from the project's own simulations.

Contents

1. The project in one page
2. Background: why memory is the bottleneck
3. How a cache works
4. What prefetching is
5. Our system - the big picture
6. The life of one load (step by step)
7. The prediction engine in detail
8. Our three innovations
9. The workloads (test inputs)
10. How we proved it works
11. Results and how to read them
12. Going further: perfect cache, slow DRAM, tuning
13. Hardware size (synthesis)
14. Every file explained
15. How to run everything
16. Demo script for the judges
17. Questions judges may ask (with answers)
18. Limitations and future work
19. Glossary

1. The project in one page

The challenge (from Micron's slides): AI models like LLMs, CNNs and recommenders are fast at math, but they waste time waiting for data from memory. Build hardware that fetches data *before* it is needed: **Watch** which addresses the program reads, **Guess** the next address, **Fetch early** into the cache. Result: fewer slow DRAM trips, more cache hits, faster AI.

What we built:

- A **cache** (4 KB, 4-way, 16-byte lines) written in Verilog - the hardware description language chips are designed in.
- A **prefetch engine** that learns the step size (stride) of up to 4 memory streams, decides how sure it is, and loads the next lines early.
- Three **adaptive tricks**: it measures its own accuracy and (1) goes further ahead when right, (2) gives short new streams a quick start, (3) protects useful data when its guesses are poor.
- A **DRAM model** (40-cycle delay) and a **CPU model** that replays memory traces.
- **9 test workloads** made in Python, including CNN-, LLM- and recommender-like patterns.
- A full **verification** flow: self-checking testbench, Python golden model, a second simulator (Verilator), Cocotb Python tests, lint and synthesis.

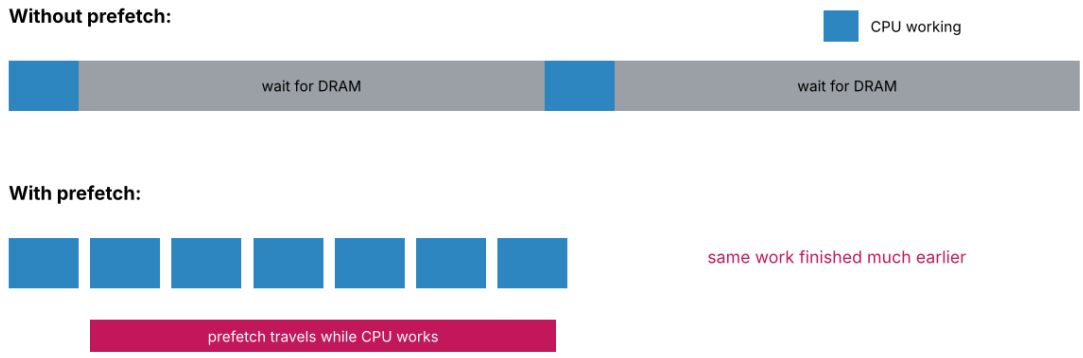
Question	Answer
How much faster?	2.26x average over 9 workloads, 2.06x on AI-inspired ones, up to 5.2x
Is it correct?	54/54 simulations pass; matches Python model exactly; identical on 2 simulators; 7/7 Cocotb tests
How big is the prefetcher?	4.9 % of the logic, 1.8 % of the flip-flops
Slower DRAM (120 cycles)?	3.03x average, 2.97x on AI-inspired workloads
Tools	Python 3, Icarus Verilog, GTKWave, Git, Verilator, Cocotb, Pandas, Matplotlib (+ Yosys)

2. Background: why memory is the bottleneck

A processor can do one calculation every clock cycle. But reading data from main memory (DRAM) takes tens to hundreds of cycles. If the processor must wait for every piece of data, it sits idle most of the time. This is called the **memory wall**.

AI workloads make it worse: a large language model reads millions of weights for every word it produces, and a recommender jumps around huge embedding tables. The math units are fast; the data supply is slow.

Simple analogy. DRAM is a big library far away. The cache is your small desk. Working from the desk is instant; walking to the library takes a long time. A prefetcher is a helper who notices which books you are reading in order and puts the next ones on your desk before you ask.



Prefetching overlaps the slow DRAM trip with useful work.

Key numbers in our model

Item	Value	Meaning
Cache hit	1 cycle	data is already in the cache
Cache miss	about 42 cycles	40-cycle DRAM delay + a few control cycles
DRAM bandwidth	1 new request every 4 cycles	so useless prefetches really cost something
CPU compute gap	1-6 cycles between loads	the 'math' the AI does between memory reads

AMAT (Average Memory Access Time) = hit time + miss rate x miss penalty. Example: with 25 % misses, AMAT = 1 + 0.25 x 41 = about 11 cycles. If prefetching cuts misses to 1 %, AMAT drops to about 1.4 cycles. That is exactly what we measured on the sequential workload (11.25 -> 1.07).

3. How a cache works

Lines

The cache does not store single bytes. It stores **lines** of 16 bytes (4 words of 32 bits). When you read one word, the whole line comes from DRAM, so the next 3 words are free hits. That is **spatial locality**. Reading the same data again soon is **temporal locality**.

Where does a line go? - the address split

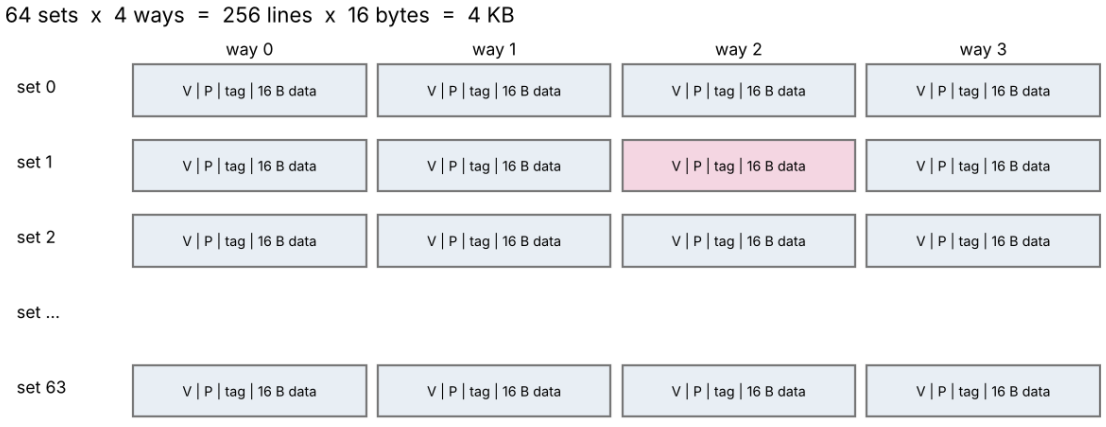


32-bit address split inside the cache

Worked example: address `0x00100A34`

- WORD = bits 3..2 = 1 (the second word in the line)
- INDEX = bits 9..4 = 35, so the line can only live in **set 35**
- TAG = bits 31..10 = 0x402. The cache stores the tag to remember *which* memory block is in that slot.

Sets and ways



V = valid bit, P = 'prefetched, not yet used' bit, plus an LRU age per way

Each set has **4 ways** (4 slots), so 4 different lines with the same index can live together. On a lookup the cache reads all 4 tags of the set at once and compares them with the address tag. Equal tag + valid bit = **hit**. Otherwise **miss**.

Which line is thrown out? - LRU

When a set is full and a new line arrives, we evict the **Least Recently Used** line. Each way has a 2-bit age (0 = just used, 3 = oldest). On a hit the line becomes age 0 and the younger ones age by one. Example: lines A, B, C, D fill a set; you touch A again; then E arrives. B is now the oldest, so B is evicted. (This exact case is one of our Cocotb tests.)

Why 4 KB, 4-way?

Option	Good	Bad
Direct-mapped (1 way)	cheapest, fastest	two streams with the same index keep evicting each other
Fully associative	no conflicts	needs 256 tag comparators - expensive
4-way set associative (chosen)	few conflicts, only 4 comparators	a bit more logic than direct-mapped

4 KB is deliberately small so that our workloads (8-60 KB) do not fit, exactly like real AI models do not fit in real caches.

4. What prefetching is

Prefetching = loading data into the cache before the program asks for it. A good prefetcher must guess **what** to fetch, **when** (early enough to arrive in time, not so early that it is evicted before use) and **how much** (too much wastes DRAM bandwidth and pushes out useful data).

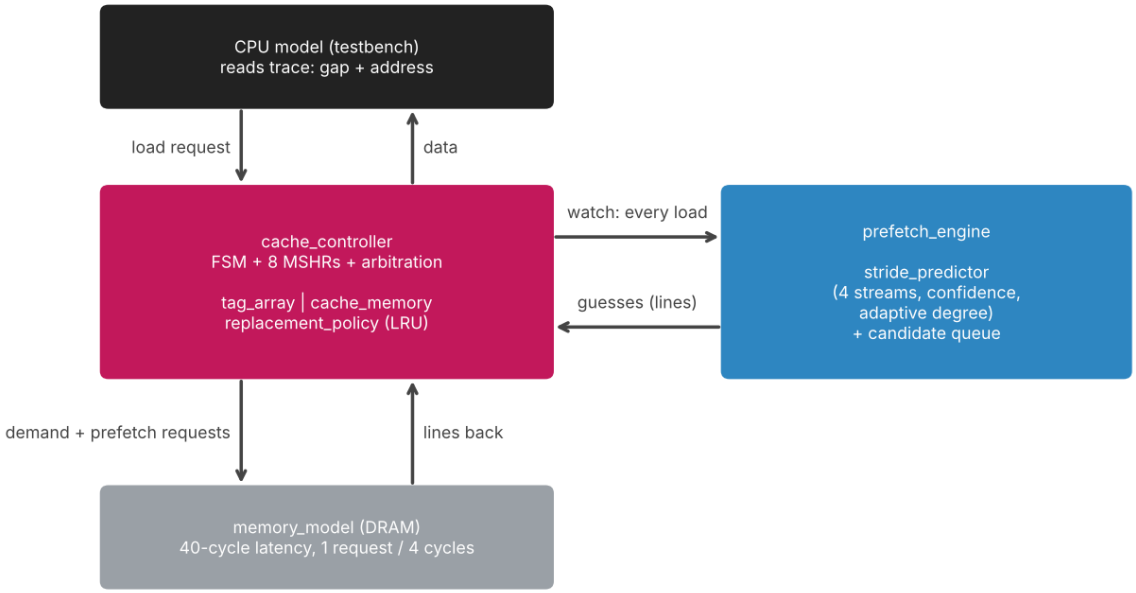
Common strategies

Strategy	Idea	Works well for	Fails on
Next-line (sequential)	after using line L, fetch L+1	simple streams, very short streams	any jump bigger than one line; wastes bandwidth on random data
Stride	learn the step between accesses (e.g. +48 B) and continue it	array walks, matrix rows, channels	short streams (needs time to learn)
Adaptive stride	stride + adjust distance/amount by measured accuracy	long regular AI streams	nothing new for random data
Pattern-based / ML	learn complex sequences	irregular repeat patterns	large and complex hardware

Measuring a prefetcher

- **Useful prefetch:** the line was used by the CPU while it was still waiting in the cache (on time).
- **Late prefetch:** the CPU asked while the line was still on its way from DRAM. It still saves part of the wait.
- **Unused prefetch:** fetched but never used - wasted DRAM bandwidth.
- **Accuracy** = useful / issued. **Coverage** = share of the baseline misses that were removed.
- Important: prefetches are *never* counted as CPU accesses, so hit rate is always about real CPU loads.

5. Our system - the big picture



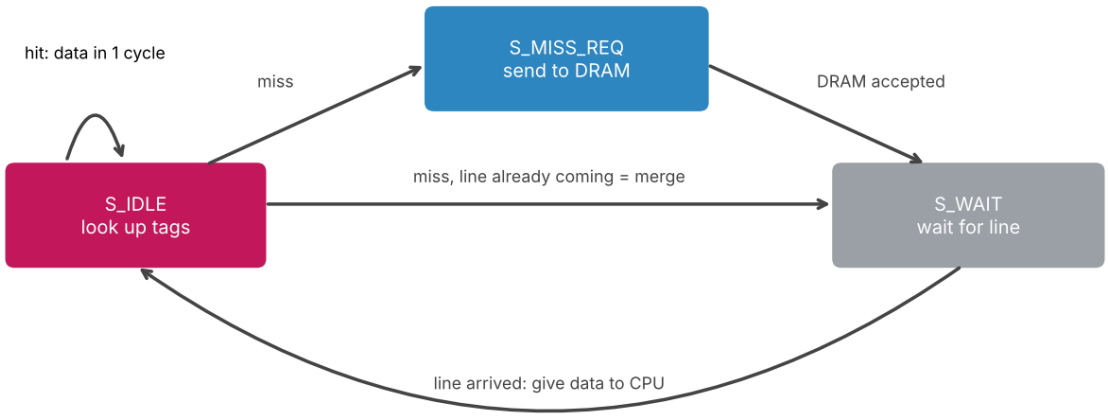
Blocks of the design. Pink and blue are the real hardware; grey is the DRAM simulation model.

Block	Job
cache_controller	the brain of the cache: looks up tags, handles hits and misses, sends requests to DRAM, decides who goes first
tag_array	stores for every line: tag, valid bit, 'prefetched' bit
cache_memory	stores the 16-byte data of every line
replacement_policy	keeps the LRU ages; picks which line to evict; can insert a line as 'oldest'
stride_predictor	the WATCH and GUESS part: stream table, confidence, triggers, accuracy monitor
prefetch_engine	turns a guess into a list of line addresses and keeps them in an 8-entry queue
memory_model	fake DRAM for simulation: 40-cycle delay, limited bandwidth, data = a formula of the address
top_module	connects everything; settings choose baseline or a prefetch mode

MSHRs - many requests in flight

An **MSHR** (Miss Status Holding Register) remembers one request that has gone to DRAM and not come back yet. We have 8. The CPU can only wait for one load at a time, but the prefetcher can keep up to 7 requests travelling at once - that is how the 40-cycle delay gets hidden. One MSHR is always kept free for the CPU.

6. The life of one load (step by step)



The demand state machine inside cache_controller.

Case A - hit

- Cycle 0: the CPU puts the address on the bus. The cache splits it, reads the 4 tags of the set and compares them.
- A tag matches: the data word is returned in the next cycle. The line becomes most recently used.
- If the line was brought by the prefetcher, the 'prefetched' bit is cleared and the 'useful' counter goes up.

Case B - miss

- No tag matches and the line is not on its way: go to S_MISS_REQ and send the request to DRAM (demand requests always go before prefetches).
- S_WAIT: about 40 cycles later the line comes back, is written into the victim way chosen by LRU, and the word goes to the CPU.

Case C - late prefetch (merge)

- No tag matches, but an MSHR shows the line is already coming because of a prefetch.
- The cache does **not** send a second request. It marks that MSHR as 'CPU is waiting' and waits only for the remaining time (our Cocotb test measured 11 cycles instead of 42).

Meanwhile - the prefetch path

- Every access is also sent to the predictor (WATCH).
- When it is confident it puts line addresses into the queue (GUESS).
- Each cycle the controller checks the queue head: already in cache or already coming? drop it. Otherwise, if the DRAM port is free and at least 2 MSHRs are free, send it (FETCH EARLY).
- When the line returns it is stored with the 'prefetched' bit set.

Rules that keep it correct

Rule	Why
Demand first on the DRAM port	prefetching must never slow down the CPU's own misses

Rule	Why
Only one write to the cache arrays per cycle (fills wait during a CPU lookup)	no two writers can corrupt the same line
One MSHR always free for the CPU	a burst of prefetches can never block a demand miss
Drop candidates already cached or in flight	no duplicate lines, no wasted DRAM trips
No prefetch across a 4 KB page	never guess into unrelated memory

7. The prediction engine in detail

The stride predictor has a table of **4 streams**. Each entry remembers: last address, stride (step in bytes) and a 2-bit **confidence** (0-3). Our traces only contain addresses (no program counter), so a load belongs to the stream whose last address is within 1 KB of it. Streams far apart (weights, activations, KV cache) get separate entries.

Learning rules

- Same stride as before -> confidence + 1 (max 3).
- Different stride -> confidence - 1; if confidence is 1 or less, learn the new stride.
- No stream matches -> start a new entry (replace a free or a confidence-0 entry first, so random loads cannot erase a good stream).
- Fire when confidence >= 2 and the load moved into a new cache line: prefetch addr + 1 x step ... addr + degree x step.
- Strides smaller than a line (e.g. 4 bytes) are rounded up to one line, so word-by-word streams fetch whole next lines.

Worked example: a 48-byte stride

Load	Address	Distance	Stream state after	Action
1	0x2000	-	new stream, stride 0, conf 0	none
2	0x2030	+48	stride learned = 48, conf 0	none
3	0x2060	+48	same stride, conf 1	none (not sure yet)
4	0x2090	+48	conf 2, new line	prefetch 0x20C0 and 0x20F0 (degree 2)
5	0x20C0	+48	conf 3	load hits (was prefetched); prefetch 0x2120 (0x20F0 is filtered as duplicate)

The degree (how many lines ahead) is not fixed - see innovation 1. On long streams it climbs to 7, so the prefetch is started 7 steps before the data is needed, which is enough to hide the 40-cycle DRAM delay.

8. Our three innovations

All three use one small **accuracy monitor**: two 6-bit counters that count, over every 32 issued prefetches, how many were used (on time or late).

Innovation 1 - accuracy-feedback adaptive degree (slide: Intelligent Prefetching)

Measured accuracy (last 32 prefetches)	Action
>= 75 % used	degree + 1 (up to 7): run further ahead
< 25 % used	degree - 1 (down to 1): back off

Effect (fixed degree 2 -> adaptive): sequential 2.75x -> 3.85x, stride 2.69x -> 5.20x, average 1.84x -> 2.21x. This is the biggest single gain.

Innovation 2 - hybrid quick start for new or jumping streams (slide: Novel Prediction Algorithms, AI-Aware)

Recommendation models read short embedding rows (64 bytes = 4 lines) at random places. The stride detector needs about 5 loads to become sure, so the row is almost finished before it helps. The same happens in a CNN each time it moves to the next feature-map row: the stream *jumps* and has to re-learn. Fix: when a **new** stream starts with a **miss**, or a known stream **jumps** on a miss, immediately prefetch the next 2 lines while the stride re-trains. To stop this from wasting bandwidth on random data, it is switched **off while measured accuracy is below 50 %**.

Effect (adding this idea): CNN 2.00x -> 2.18x, recsys 1.34x -> 1.37x, repeated 1.00x -> 1.08x. On pure random data only 32 prefetches are issued in total, while a plain next-line prefetcher doubles DRAM traffic. The jump case was found by logging every slow CNN load and seeing that the first two lines of each new row always missed.

Innovation 3 - adaptive insertion (slide: Adaptive Cache Policies)

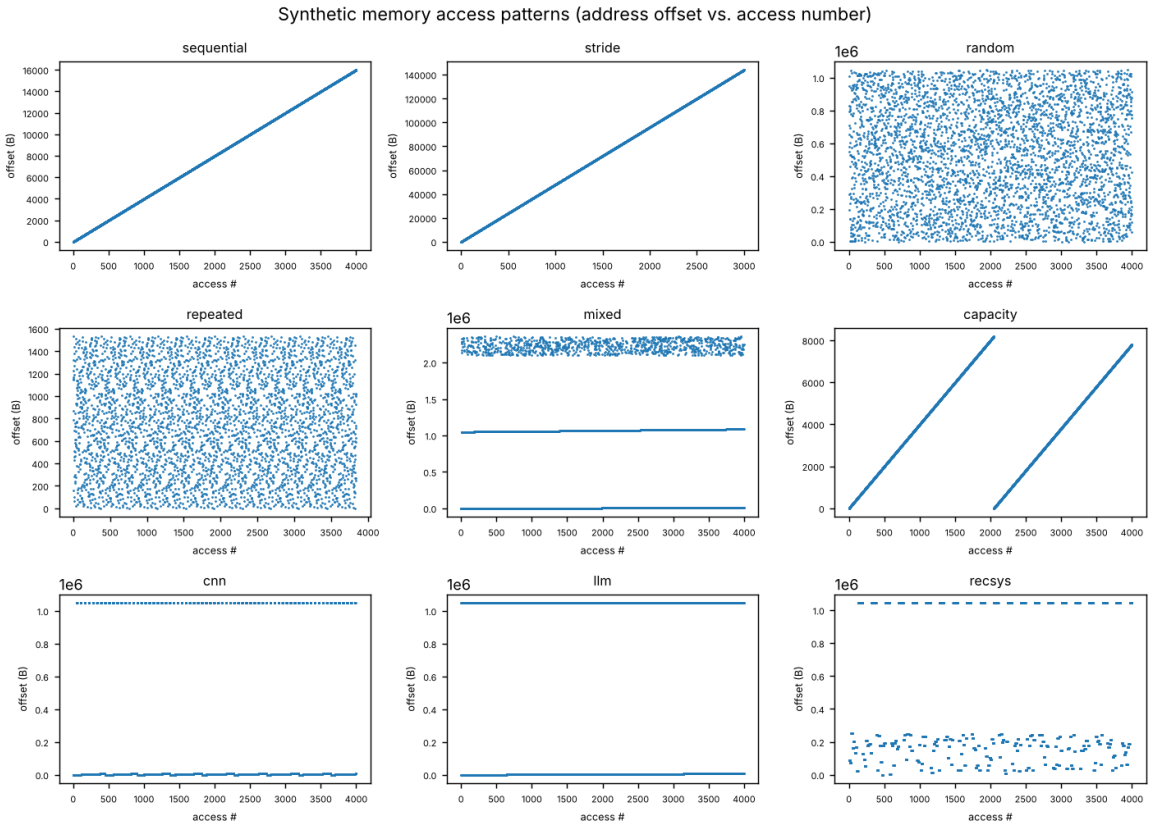
When guesses are poor (accuracy below 50 %), new prefetched lines are inserted as the **oldest** line in their set, so a wrong guess is the first thing thrown out and cannot push out useful data. When guesses are good they are inserted as newest, so they survive until used. We first tried 'always insert as oldest' and it hurt multi-stream workloads - that measurement is why the policy is adaptive.

Effect: recsys 1.37x -> 1.46x with 9 % less DRAM traffic. Honest cost: mixed 2.19x -> 2.11x.

9. The workloads (test inputs)

Micron gave no traces, so `python/generate_traces.py` creates them. Each line of a trace is '*compute gap address*'. The same fixed random seed is used every time, so results are reproducible. These are **synthetic models** of access patterns, not recordings of real AI programs.

Trace	Loads	What it imitates
sequential	8192	reading a 32 KB tensor from start to end
stride	3000	every 48th byte (not line-aligned) - e.g. one field of a structure array
random	4000	random words in 1 MB - nothing to predict
repeated	3840	a 1.5 KB working set read 10 times in the same shuffled order (fits in cache)
mixed	6000	two streams + 25 % random loads, interleaved
capacity	6144	an 8 KB array (2x the cache) read 3 times - LRU keeps throwing out what comes next
cnn	8560	3x3 convolution reading input rows + reused weights, then a 1x1 conv walking channels (256 B stride)
llm	8320	matrix-vector product: weights streamed once, activation vector reused; then KV-cache scans
recsys	9600	embedding lookups: 8 rows per sample, popular rows reused (Zipf), each row 4 lines; plus a small MLP



Address vs. time for every workload (from analyze_traces.py).

10. How we proved it works

Wrong hardware that looks fast is worthless, so correctness was checked in five independent ways.

Check	How	Result
1. Self-checks in every simulation	testbench checks every returned word against the DRAM formula; hits+misses = loads; no line stored twice; every prefetch is accounted for (used, late, evicted unused, or still in cache); all MSHRs empty at the end	54/54 runs PASS
2. Golden model	a separate Python LRU cache computes hits/misses for the baseline; the RTL must match exactly	9/9 traces: exact match
3. Second simulator	the same testbench is compiled with Verilator and all 54 runs repeated; every number compared	54/54 identical
4. Cocotb tests	7 Python tests poke the hardware directly (see below)	7/7 PASS
5. Lint	Verilator -Wall looks for latches, width errors, multiple drivers	0 warnings

The 7 Cocotb tests

Test	What it proves	Measured
hit_and_miss_latency	first load misses, rest of the line hits	miss 42 cycles, hit 1 cycle
lru_replacement	the least recently used line is evicted	pass
random_vs_golden_model	1500 random loads, hits/misses equal the Python model	554 hits / 946 misses, exact
prefetch_removes_misses	a 128-line stream	demand DRAM requests 128 -> 1
demand_during_prefetch	a load to a line on its way merges; an unrelated miss is not delayed	merge 11 cycles, miss 42 cycles with 3 prefetches in flight
no_prefetch_across_page	never guesses into the next 4 KB page	30 prefetches, 0 across
random_traffic_is_throttled	accuracy gate stops waste on random data	32 prefetches for 600 loads

Honesty note: two Cocotb tests failed on their first run because the *tests* were wrong (one counted late prefetches as full misses, one loaded a line that had already arrived). The tests were fixed; the hardware was not changed.

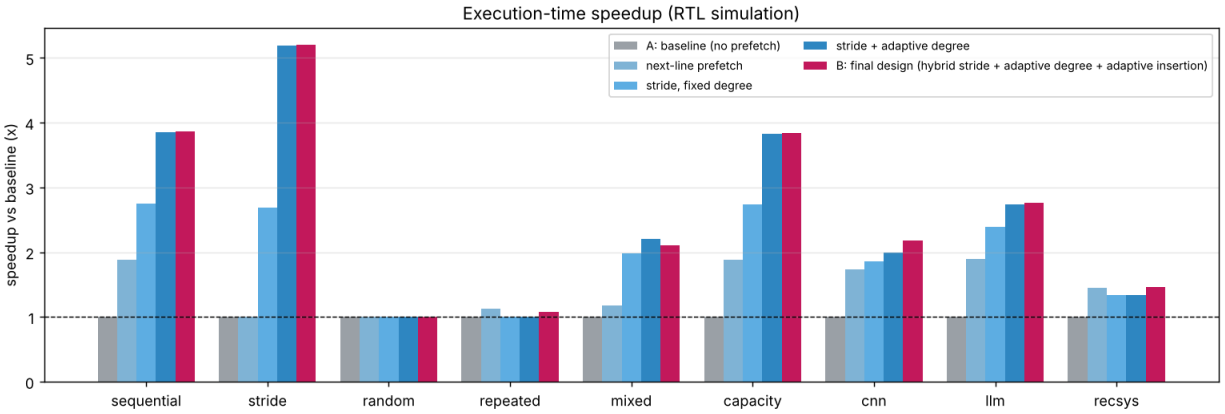
GTKWave waveforms

`simulation/waveforms/demo_final.vcd` is a short run you can open in GTKWave. You can see prefetch requests (pf_issue) firing in the gaps between CPU loads and never in the same cycle as a CPU request, several MSHRs busy at once (m_v), duplicates dropped (pf_drop), and later loads hitting in 1 cycle. In this demo the baseline needs 26 demand DRAM requests; the final design needs 3.

11. Results and how to read them

Both architectures use the **same** RTL, traces and DRAM timing - only the prefetch setting changes. A = baseline cache (no prefetch). B = final design.

Trace	Hit % A -> B	AMAT A -> B (cycles)	Cycles A -> B	Speedup	DRAM traffic
sequential	75.0 -> 99.4	11.25 -> 1.07	112571 -> 29172	**3.86x**	1.00x
stride	0.0 -> 21.1	42.00 -> 6.04	133557 -> 25688	**5.20x**	1.00x
random	0.3 -> 0.3	41.89 -> 41.89	177688 -> 177688	**1.00x**	1.01x
repeated	97.5 -> 98.3	2.02 -> 1.71	17357 -> 16127	**1.08x**	1.02x
mixed	34.5 -> 72.8	27.83 -> 11.89	182100 -> 86455	**2.11x**	1.44x
capacity	75.0 -> 99.1	11.25 -> 1.09	84464 -> 22016	**3.84x**	1.00x
cnn	80.2 -> 95.8	9.13 -> 2.02	112549 -> 51680	**2.18x**	1.45x
llm	84.4 -> 99.5	7.41 -> 1.10	82323 -> 29855	**2.76x**	1.03x
recsys	88.3 -> 93.8	5.78 -> 3.17	79479 -> 54379	**1.46x**	1.79x
average				**2.26x**	



Speedup of every architecture we compared.

What the results mean

- **Long regular streams** (sequential, capacity, LLM): almost every miss disappears and DRAM traffic stays the same - the same lines are just fetched earlier.
- **stride**: 5.2x faster even though most prefetches are 'late': the CPU wants a new line every ~3.5 cycles but DRAM accepts one only every 4, so this run is limited by DRAM bandwidth.
- **random**: nothing can be predicted; the design correctly stays quiet (1.00x, almost no extra traffic).
- **repeated**: everything already fits in the cache; only the 96 first-touch misses can be helped (1.08x).
- **CNN, recsys, mixed**: real speedups, paid for with more DRAM traffic (about 1.4-1.8x).

Comparing all strategies (average speedup)

Architecture	All 9 traces	AI-inspired (cnn, llm, recsys)
baseline	1.00x	1.00x
nextline	1.41x	1.68x
stride	1.84x	1.81x

Architecture	All 9 traces	AI-inspired (cnn, llm, recsys)
stride_adapt	2.21x	1.94x
smart_noins	2.25x	2.02x
smart	2.26x	2.06x

smart = final design. Each row adds one idea to the row above, which shows how much every idea contributes.

12. Going further: perfect cache, slow DRAM, tuning

How close are we to a perfect cache?

A 'perfect' cache would answer every load in 1 cycle. Because the CPU model is the same, its run time can be computed exactly from the baseline run. This tells us how much of the *possible* gain we achieved.

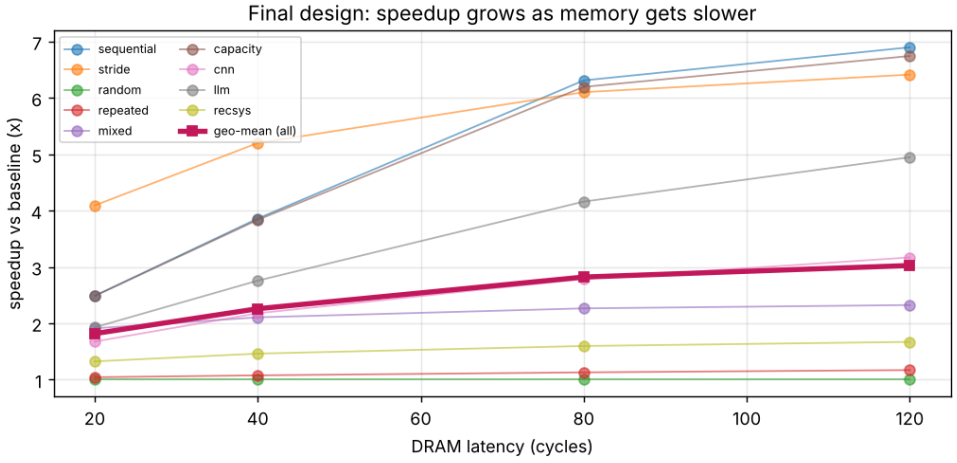
Trace	Speedup now	Max possible	Share of possible gain
sequential	3.86x	3.94x	99 %
stride	5.20x	12.65x	88 %
random	1.00x	12.57x	0 %
repeated	1.08x	1.29x	31 %
mixed	2.11x	8.63x	59 %
capacity	3.84x	3.93x	99 %
cnn	2.18x	2.62x	87 %
llm	2.76x	2.84x	98 %
recsys	1.46x	2.37x	55 %

Regular streams (sequential, capacity, LLM) reach 98-99 % of perfect: nothing left to win. Random addresses cannot be predicted by anyone (0 %), which also limits mixed and recsys. stride is limited by DRAM bandwidth.

What if DRAM is slower?

Real DRAM is usually more than 100 CPU cycles away. We re-ran everything with 20, 40, 80 and 120-cycle DRAM (only the DRAM model changes):

DRAM latency	Average speedup (9 traces)	AI-inspired (cnn, llm, recsys)
20 cycles	1.82x	1.62x
40 cycles	2.26x	2.06x
80 cycles	2.82x	2.65x
120 cycles	3.03x	2.97x



The slower the memory, the more the prefetcher helps.

Is the design tuned to death? - design-space exploration

We tried 16 combinations of its settings (stream-table size, confidence threshold, quick-start length, accuracy window). All 16 give between 2.22x and 2.24x: the design is robust, not fragile. The best one was under 1 % better and was tuned on the same traces, so we kept the simple defaults (no overfitting).

An idea we tested and rejected

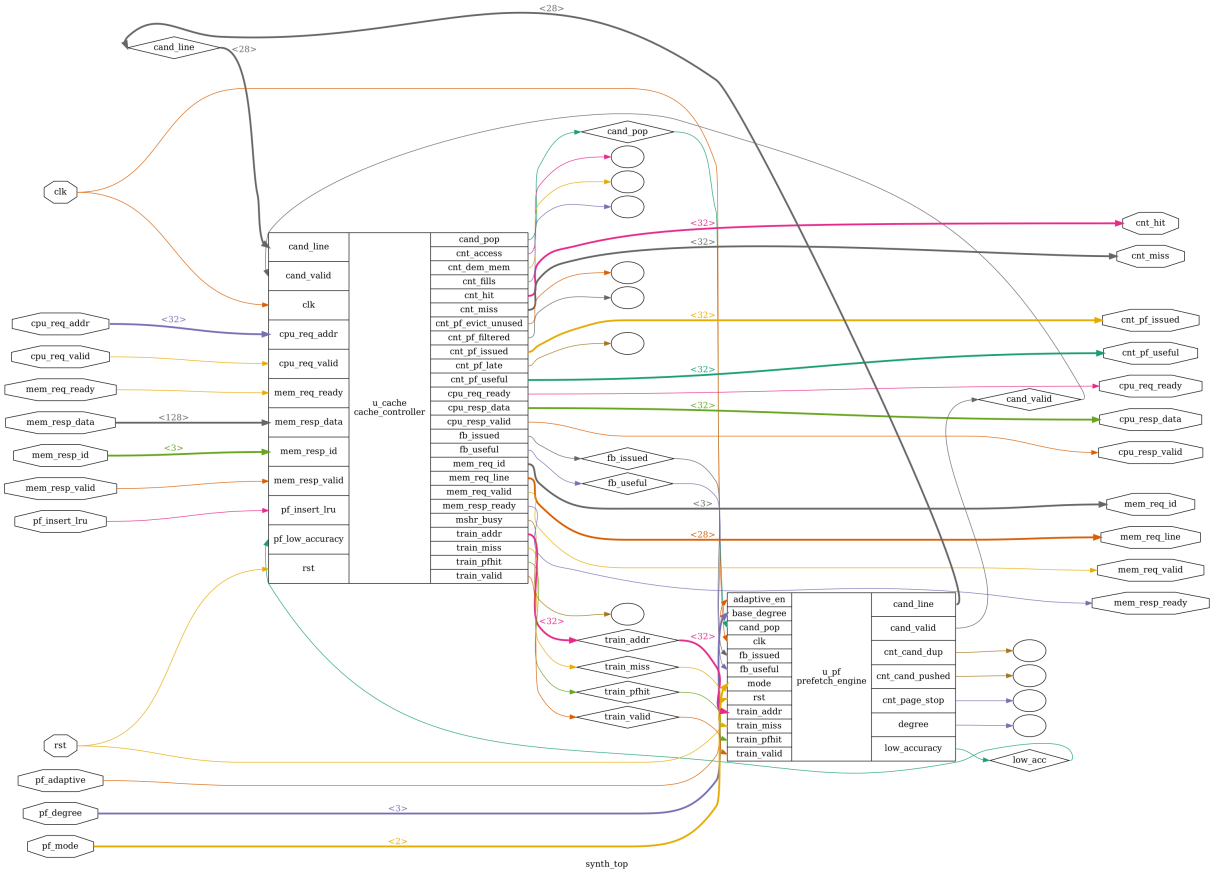
CNN channels 256 bytes apart all fall into the same 4 cache sets. Mixing higher address bits into the set number (XOR hashing) made CNN 3.4 % faster but recsys 1.3 % slower, and it changes the baseline cache too, so it was not adopted. Testing ideas and keeping only clear wins is part of good engineering.

13. Hardware size (synthesis)

Yosys (a free synthesis tool) turned the Verilog into logic gates. This shows the design is real, buildable hardware, and how much each part costs.

Block	Logic cells	Flip-flops
cache data (4 KB)	65,968	32,768
tags + valid + prefetched bits	22,953	6,144
cache controller	3,872	634
LRU replacement	1,855	512
prefetch engine (predictor + queue)	**4,827 (4.9 %)**	**733 (1.8 %)**
total	**99,471**	**40,791**

The 'intelligence' is under 5 % of the hardware; almost everything is the cache storage itself. These are generic gates - no chip area, clock speed, power or placement is claimed.



Yosys drawing of how the cache controller and prefetch engine are wired.

14. Every file explained

File / folder	What it is
rtl/top_module.v	connects cache, prefetch engine and DRAM; run-time settings pf_mode (0 base, 1 next-line, 2 stride, 3 final), adaptive, degree, insertion
rtl/cache_controller.v	demand FSM, MSHR table, prefetch probe/filter, arbitration, performance counters
rtl/tag_array.v, cache_memory.v	tag/valid/prefetched storage (3 read ports) and data storage
rtl/replacement_policy.v	true LRU with MRU or LRU insertion
rtl/stride_predictor.v	stream table, confidence, triggers, adaptive degree, accuracy flag, hybrid quick start
rtl/prefetch_engine.v	candidate generator (page-bounded), duplicate filter, 8-entry queue
rtl/memory_model.v	DRAM simulation model (not part of the design)
testbench/tb_system.v	CPU model reading a trace + all self-checks; prints RESULT line and TEST PASSED/FAILED
testbench/cocotb/test_cache.py	the 7 Cocotb tests
python/generate_traces.py	makes the 9 workloads + demo trace
python/analyze_traces.py	locality/stride statistics and access-pattern plot
python/ref_cache_model.py	golden Python LRU cache
python/run_all.py	compiles RTL, runs 54 simulations, golden check, computes all metrics
python/verilator_crosscheck.py	re-runs everything on Verilator and compares
python/run_cocotb.py	runs the Cocotb tests
python/visualize_results.py, evaluate_results.py	plots and report tables (Matplotlib, Pandas)
synth/	Yosys synthesis scripts, block diagrams, gate counts
results/	performance_summary.csv, comparison.txt, logs, plots, lint, cross-check, Cocotb results
docs/	architecture.md, algorithm.md, final_report.md, this guide
run_all.sh	one command that reproduces everything

15. How to run everything

In Ubuntu (WSL), with the tools installed as described in README.md:

```
cd ~/micron_cache_prefetch
bash run_all.sh                # traces, lint, 54 sims, Verilator check, Cocotb, plots (~1 min)

# look at the key lines again
grep -E "PASSED|MATCH" results/regression_log.txt
cat results/verilator_crosscheck.txt | tail -1
grep TESTS results/cocotb_log.txt

# one single run (mode 0 = baseline, 3 = final design)
iverilog -g2005 -o simulation/build/sim.vvp testbench/tb_system.v rtl/*.v
vvp -n simulation/build/sim.vvp +trace=traces/llm.trace +mode=0
vvp -n simulation/build/sim.vvp +trace=traces/llm.trace +mode=3 +adaptive=1 +degree=2 +inslru=1

# pictures and waveform
explorer.exe "$(wslpath -w results/plots/speedup.png)"
gtkwave simulation/waveforms/demo_final.vcd simulation/waveforms/demo.gtkw &

# hardware view
cd synth && bash run_synth.sh
```

Reading a RESULT line: *hits, misses* = CPU loads; *dem_mem* = DRAM requests caused by misses; *pf_issued* = prefetches sent; *pf_useful* = used on time; *pf_late* = used while still travelling; *cycles* = total run time; *errors* must be 0.

16. Demo script for the judges (about 5 minutes)

Step	Show	Say
1	slide 1 / section 1 of this guide	AI waits for memory; we built a cache that guesses and fetches early
2	bash run_all.sh running	54 simulations, golden model match, identical on two simulators, 7 Cocotb tests
3	speedup.png	2.26x average, 2.06x on AI-like workloads, never slower
4	GTKWave demo_final.vcd	prefetches fire between CPU loads; demand always first; later loads hit in 1 cycle
5	section 8	three adaptive ideas driven by one accuracy monitor
6	synthesis table	prefetcher is under 5 % of the hardware
7	limitations	synthetic traces, read-only cache - we say it openly

17. Questions judges may ask (with answers)

Q: Why did you choose 4-way set associative?

A: It removes most conflict misses between interleaved AI streams while needing only 4 tag comparators. Fully associative would need 256.

Q: How do you know your results are not just a simulator effect?

A: The same testbench gives identical numbers on two different simulators (Icarus and Verilator), and the baseline matches an independent Python model exactly.

Q: How does the prefetcher avoid slowing down normal loads?

A: Demand requests always win the DRAM port, one MSHR is always kept for the CPU, and fills never happen in the same cycle as a CPU lookup. The Cocotb test shows an unrelated miss still takes the normal 42 cycles with 3 prefetches in flight.

Q: What happens on random data?

A: Confidence never builds, and the quick-start is switched off by the accuracy gate, so only 32 prefetches are issued over 4000 random loads; speed is unchanged.

Q: What is a late prefetch, and is it bad?

A: The CPU asked while the line was still on its way. It is counted as a miss, but the CPU waits only the remaining time and no second DRAM request is sent, so it still helps.

Q: Where does the prefetcher lose?

A: It costs extra DRAM traffic on CNN and recsys (about 1.4-1.8x), and the adaptive insertion costs 3.8 % on the mixed trace. Both are shown in the results.

Q: Why no program counter (PC) in the predictor?

A: Our traces contain only addresses, so streams are separated by address region (1 KB window). A PC-indexed table is listed as future work.

Q: Is this real hardware?

A: Yes: synthesizable Verilog, lint clean, and it synthesizes with Yosys to about 99k gates. We do not claim area or clock speed because no real chip library was used.

Q: Is it really 'AI-aware'?

A: It adapts to the kind of traffic it sees: long regular streams (CNN/LLM) push it to run further ahead, short irregular bursts (recommenders) and stream jumps (new CNN rows) trigger the quick start, poor accuracy makes it cautious. The traces are synthetic models of these patterns, not real model runs.

Q: How long did it take to run everything?

A: The whole flow (traces, 54 simulations, Verilator check, Cocotb, plots) runs in under a minute.

18. Limitations and future work

Limitation	Future improvement
synthetic traces, not real AI programs	replay traces captured from real model runs

Limitation	Future improvement
read-only cache (no stores)	add write-back with dirty bits
one cache level, CPU with one outstanding load	multi-level hierarchy, out-of-order CPU model
streams matched by address region	PC-indexed stride table
simple fixed-latency DRAM	banks, rows and refresh
generic-gate synthesis only	SRAM macros + OpenROAD for real area, timing and placement
global accuracy for all streams	per-stream accuracy and degree

19. Glossary

Term	Meaning
AMAT	average memory access time = hit time + miss rate x miss penalty
Bandwidth	how many requests DRAM can accept per unit of time
Cache line	the 16-byte block the cache moves at once
Cocotb	a Python library for writing hardware tests
Confidence	how sure the predictor is about a stream's stride (0-3)
Coverage	share of baseline misses removed by prefetching
Degree	how many lines ahead the prefetcher fetches (1-7, adaptive)
Demand access	a load the CPU actually asked for (not a prefetch)
DRAM	main memory: large but slow
FSM	finite-state machine: hardware that moves between states (IDLE, MISS_REQ, WAIT)
Golden model	an independent reference implementation used to check the hardware
GTKWave	a viewer for signal waveforms
Hit / miss	data found / not found in the cache
Icarus Verilog	free Verilog simulator (main simulator here)
Index / tag	address bits that choose the set / identify the block
Late prefetch	the CPU asked while the prefetched line was still on its way
Lint	automatic check of code for suspicious constructs
LRU	least recently used - the line evicted when a set is full
MSHR	register that tracks one request still waiting for DRAM
Prefetch	fetching data into the cache before it is asked for
RTL	register-transfer level: the hardware description in Verilog
Set / way	a group of slots chosen by the index / one slot in that group
Stride	the constant step between addresses of a stream
Synthesis	turning Verilog into logic gates (Yosys)
Trace	a list of memory addresses a program reads, used as test input
Verilator	a second, very fast Verilog simulator and lint tool
Yosys	free synthesis tool